CepteCüzdan – AI Destekli Akıllı Bütçe Asistanı Proje Ön Raporu

Mobil Programlama Dersi · Ekip Üyeleri: Mehmet Demir (02230202050), Berkay Başaran (02230202058)

Özet

CepteCüzdan, gelir-gider takibini form doldurma zahmetinden kurtaran, Flutter ile geliştirilen ve verileri cihazda SQLite ile saklayan bir mobil bütçe asistanıdır. Kullanıcı harcamasını fiş fotoğrafı çekerek, mikrofona konuşarak ya da chatbot'a yazarak ekler. Yapay zeka tutarı, tarihi, işletmeyi ve kategoriyi çıkarır, kaydetmeden önce kullanıcıdan onay ister.

Uygulama dört modülden oluşur: Ana Sayfa (Dashboard), Chatbot ve Asistan, Manuel İşlem, Raporlama ve Ayarlar. Proje, Mobil Programlama dersi kapsamında iki kişilik bir ekip tarafından geliştirilmektedir.

Giriş ve Problem Tanımı

Kişisel bütçe uygulamalarının çoğu, kullanıcının her harcamayı tutar, tarih ve kategori alanları olan bir formla elle girmesini bekler. Bu yöntem ilk günlerde işe yarasa da küçük ve sık harcamalar biriktikçe kayıt alışkanlığı kaybolur. Sonuçta veriler eksik kalır ve uygulamanın gösterdiği tablo gerçeği yansıtmaz.

Sorunun temelinde üç sürtünme noktası vardır:

- **Zaman:** Market fişindeki tutarı ve tarihi tek tek yazmak, harcama anında kullanıcıya yük olur.
- **Bilişsel yük:** Her harcama için doğru kategoriyi seçmek karar vermeyi gerektirir ve ertelenir.
- **Bağlam:** Taksi, kahve gibi fişsiz harcamalar telefon açılıp form doldurulana kadar unutulur.

CepteCüzdan bu sürtünmeyi kaldırmak için formu konuşmayla değiştirir. Kullanıcı bir fotoğraf çeker ya da tek cümle söyler; geri kalanı asistan doldurur ve yalnızca onay ister.

Projenin Amacı ve Hedefleri

Projenin amacı, gelir-gider kaydını birkaç saniyelik bir onay adımına indiren ve kullanıcıya bütçesinin anlık resmini gösteren bir mobil asistan geliştirmektir. Bu amaca ulaşmak için aşağıdaki hedefler belirlenmiştir:

1. Fiş fotoğrafından işletme adını, toplam tutarı, tarihi ve mümkün olduğunda ürün kalemlerini otomatik okumak.
2. "Taksiye 150 lira verdim" gibi doğal dildeki sesli veya yazılı ifadelerden tutar ve kategori çıkarmak.
3. Yapay zekanın çıkardığı her kaydı, veritabanına yazmadan önce chatbot arayüzünde kullanıcının onayına sunmak.
4. Aylık kalan bütçeyi, gelir-gider dengesini ve kategori dağılımını grafiklerle tek ekranda göstermek.
5. Yapay zeka özelliklerine erişilemediğinde de klasik form ile işlem eklemeyi mümkün kılmak.
6. Kategori bazlı haftalık ve aylık raporlar üretmek, belirlenen bütçe limitleri aşıldığında kullanıcıyı uyarmak.
7. Finansal verileri cihazda SQLite ile saklayarak kullanıcı gizliliğini korumak.

Kapsam

Proje tek kullanıcılı, cihaz üzerinde çalışan bir Android/iOS uygulaması olarak sınırlandırılmıştır. Kapsamı netleştirmek için dahil olan ve bu sürümde yer almayan özellikler aşağıda ayrılmıştır.

Kapsam içinde	Kapsam dışında
Fiş fotoğrafından tutar, tarih ve işletme okuma	Banka hesabı veya kredi kartı entegrasyonu
Sesli ve yazılı komutla harcama ekleme	Birden fazla kullanıcı arasında ortak bütçe
Chatbot üzerinden onaylı kayıt	Bulut senkronizasyonu ve kullanıcı hesabı
Manuel gelir/gider ekleme, düzenleme, silme	Döviz kuru takibi ve çoklu para birimi
Kategori yönetimi (ekleme, renk, ikon)	Yatırım ve borç takibi
Dashboard grafikleri ve haftalık/aylık raporlar	Web veya masaüstü sürümü
Kategori bazlı bütçe limiti ve aşım uyarısı	

Kullanılan Teknolojiler

Uygulama Flutter (Dart) ile tek kod tabanından Android ve iOS için geliştirilir; veriler cihazda SQLite'ta tutulur. Yapay zeka işleri iki aşamaya bölünür: metin cihazda çıkarılır, yalnızca anlamlandırma adımı için Gemini modeline gönderilir.

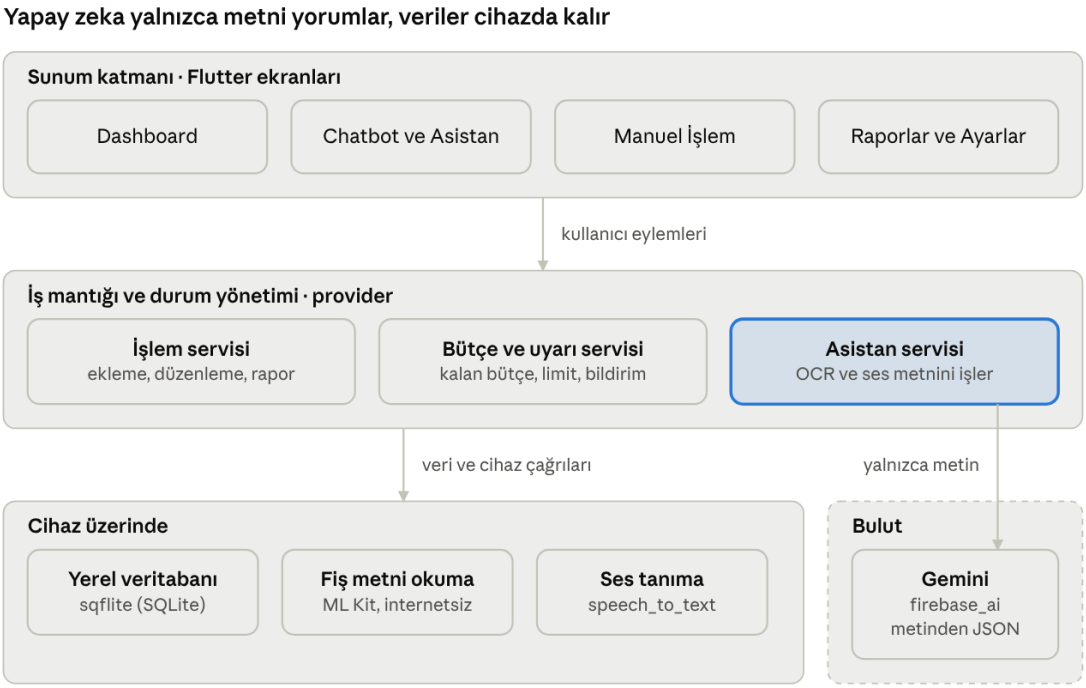
İhtiyaç	Teknoloji / Paket	Görevi
Arayüz ve uygulama çatısı	Flutter (Dart)	Tek kod tabanından Android ve iOS arayüzü
Yerel veritabanı	sqflite (SQLite)	İşlem, kategori, bütçe ve sohbet geçmişini cihazda saklama
Grafikler	fl_chart	Dashboard'daki pasta grafik ve raporlardaki çubuk/çizgi grafikler
Fiş fotoğrafı	image_picker	Kamera veya galeriden fiş görüntüsü alma
Fiş metni okuma (OCR)	google_mlkit_text_recognition	Fişteki metni internet gerektirmeden cihazda çıkarma (Latin alfabesi, Türkçe dahil)
Sesli komut	speech_to_text	Cihazın kendi konuşma tanıma servisiyle sesi Türkçe metne çevirme
Doğal dil anlama	firebase_ai (Firebase AI Logic, Gemini)	OCR veya konuşma metninden tutar, tarih, işletme ve kategori çıkarıp JSON döndürme
Durum yönetimi	provider	Ekranlar arasında bütçe ve işlem verisini paylaşma

İhtiyaç	Teknoloji / Paket	Görevi
Bildirimler	flutter_local_notifications	Bütçe aşım uyarılarını yerel bildirim olarak gösterme
İzinler ve biçimlendirme	permission_handler, intl	Kamera/mikrofon izni; TL ve Türkçe tarih biçimi

Gemini erişimi için Google'ın eski `google_generative_ai` paketi yerine `firebase_ai` tercih edilmiştir; eski paket kullanımdan kaldırılmış ve yerini Firebase SDK'ya bırakmıştır.

Sistem Mimarisi

Uygulama, ekranların veriye doğrudan erişmediği üç katmanlı bir yapıyla tasarlanmıştır: sunum, iş mantığı ve veri/cihaz servisleri.



katmanlı mimari · 3 katman, 1 bulut servisi

Ekranlar yalnızca provider üzerinden servislerle konuşur. Asistan servisi OCR ve ses tanımayı cihazda çalıştırır, Gemini'ye yalnızca ortaya çıkan metni gönderir ve dönen JSON'u onay kartına dönüştürür.

Uygulama Modülleri

Uygulama alt gezinme çubuğundan erişilen dört ana modülden oluşur. Tüm modüller aynı SQLite veritabanını kullandığı için bir modülde eklenen işlem diğerlerinde anında görünür.

1. Ana Sayfa (Dashboard)

Kullanıcının uygulamayı açtığında ilk gördüğü, bütçenin o anki durumunu özetleyen ekrandır.

- Ayın toplam geliri, toplam gideri ve kalan bütçe kartları
- fl_chart ile kategori bazlı harcama dağılımını gösteren pasta grafik
- Gelir-gider dengesini gösteren ilerleme çubuğu veya kıyas grafiği
- Son eklenen işlemlerin kısa listesi
- Limitine yaklaşan kategoriler için görsel uyarı rozeti

2. Chatbot ve Asistan Ekranı

Uygulamanın ayırt edici modülüdür; kullanıcı harcamalarını burada yazarak, konuşarak veya fiş fotoğrafıyla ekler.

- Mesaj balonlarından oluşan sohbet arayüzü ve kalıcı sohbet geçmişi
- Kamera ve mikrofon düğmeleri ile metin giriş alanı
- Yapay zekanın çıkardığı kaydı gösteren onay kartı: tutar, işletme, tarih, önerilen kategori
- Kart üzerinde **Kaydet**, **Düzenle** ve **İptal** seçenekleri
- "Bu ay markete ne kadar harcadım?" gibi sorulara veritabanından hesaplanan yanıtlar

3. Manuel İşlem Ekranı

Yapay zekanın kullanılamadığı veya kullanıcının tercih etmediği durumlar için klasik giriş yöntemidir.

- Gelir/gider seçimi, tutar, tarih, kategori ve açıklama alanları olan form
- Mevcut işlemleri listeleme, düzenleme ve silme
- Kategori yönetimi: yeni kategori ekleme, ad, renk ve ikon belirleme
- Varsayılan kategoriler: Market, Ulaşım, Faturalar, Yeme-İçme, Eğlence, Sağlık, Diğer

4. Raporlama ve Ayarlar

Kullanıcının geçmişe dönük harcamalarını incelediği ve uygulamayı kişiselleştirdiği bölümdür.

- Haftalık ve aylık döküm: kategori toplamları ve önceki dönemle kıyas
- Tarih aralığı ve kategoriye göre filtreleme
- Aylık genel bütçe ve kategori bazlı limit tanımlama
- Limitin %80'ine ulaşıldığında ve aşıldığında yerel bildirim
- Tema (açık/koyu), para birimi görünümü ve verileri sıfırlama ayarları

Öne Çıkan Özellik: Akıllı Fiş ve Sesli Harcama Tanıma

Fiş, ses ve yazı olmak üzere üç giriş yolu aynı metin işleme hattında birleşir ve her kayıt kullanıcı onayından geçer.



fiş ve sesli harcama akışı · 3 giriş, 1 onay adımı

Fiş ve ses önce cihazda metne çevrilir; yazılı mesaj doğrudan anlamlandırma adımına girer. Modelden dönen sonuç onay kartında gösterilir; kullanıcı alanları düzenleyebilir veya kaydı iptal edebilir.

Örnek senaryolar

1. **Fiş:** Kullanıcı marketten çıkınca fişinin fotoğrafını çeker. Asistan "Migros'tan 320 TL harcama yaptın, 'Market' kategorisine kaydedeyim mi?" diye sorar; onaylanınca işlem ve okunan ürün kalemleri kaydedilir.
2. **Ses:** Kullanıcı mikrofona basıp "Taksiye 150 lira verdim" der. Asistan tutarı 150 TL, kategoriyi Ulaşım, tarihi bugün olarak önerir.
3. **Soru:** Kullanıcı "Bu ay markete ne kadar harcadım?" diye yazar. Asistan toplamı veritabanından hesaplayıp yanıtlar.

Modelden beklenen yanıt

Modele, kullanıcının kategori listesi ve bugünün tarihiyle birlikte yalnızca aşağıdaki yapıda JSON döndürmesi talimatı verilir. Tarih okunamazsa bugünün tarihi, kategori belirsizse "Diğer" kullanılır.

```
{
  "type": "expense",
  "amount": 320.00,
  "merchant": "Migros",
  "date": "2026-11-08",
  "category": "Market",
  "items": []
}
```

`items` alanı fişten okunan ürün kalemleriyle doldurulur, sesli girişte boş kalır. Uygulama yanıtı bu yapıya göre doğrular; uymayan yanıtlar onay kartına düşmez ve kullanıcıdan tekrar denemesi istenir.

Veritabanı Tasarımı

Veriler cihazdaki tek bir SQLite dosyasında beş tabloda tutulur; merkezde `transactions` tablosu yer alır. Tutarlar, ondalıklı sayı yuvarlama hatalarını önlemek için kuruş cinsinden tam sayı (INTEGER) olarak saklanır.

Tablo	Temel alanlar	Açıklama
`categories`	id, name, type (gelir/gider), color, icon, is_default	Varsayılan ve kullanıcının eklediği kategoriler
`transactions`	id, type, amount, category_id, date, merchant, note, source, receipt_path, created_at	Tüm gelir ve gider kayıtları; `source` kaydın manuel, fiş, ses veya sohbetten geldiğini belirtir
`receipt_items`	id, transaction_id, name, quantity, price	Fişten okunan ürün kalemleri
`budgets`	id, category_id, month, limit_amount	Aylık genel bütçe (category_id boş) veya kategori limiti
`chat_messages`	id, role, content, transaction_id, created_at	Sohbet geçmişi; onaylanan mesaj ilişkili işleme bağlanır

Tablolar arası ilişkiler şu şekildedir: bir kategori çok sayıda işlem ve bütçe limiti içerebilir; bir işlem sıfır ya da daha fazla fiş kalemine sahip olabilir. Tarih ve kategoriye göre yapılan rapor sorgularını hızlandırmak için `transactions(date)` ve `transactions(category_id)` alanlarına indeks eklenir.

Fonksiyonel Olmayan Gereksinimler

Uygulamanın kullanışlı sayılması için işlevlerin yanı sıra aşağıdaki kalite ölçütlerini de karşılaması hedeflenir.

- **Gizlilik:** Finansal veriler yalnızca cihazda tutulur. Fiş görüntüsü sunucuya gönderilmez; modele yalnızca OCR veya konuşma metni iletilir.
- **Çevrimdışı çalışma:** Dashboard, manuel işlem ve raporlar internet olmadan çalışır. Bağlantı yoksa asistan kullanıcıyı manuel ekrana yönlendirir.
- **Kullanıcı kontrolü:** Yapay zekanın çıkardığı hiçbir kayıt onay alınmadan veritabanına yazılmaz.
- **Performans:** Uygulama açılışında Dashboard verisi tek sorguda hazırlanır; uzun süren yapay zeka çağrılarında yükleniyor göstergesi görünür.
- **Güvenlik:** API anahtarı kaynak koda gömülmez; Firebase yapılandırması ve App Check ile yetkisiz kullanım sınırlanır.
- **Kullanılabilirlik:** Arayüz Türkçedir, tutarlar TL ve Türkçe tarih biçimiyle gösterilir, açık ve koyu tema desteklenir.
- **Bakım kolaylığı:** Arayüz, iş mantığı ve veri katmanları ayrı tutulur; yapay zeka servisi tek bir arabirim arkasında soyutlanır.

Riskler ve Çözüm Yaklaşımları

En büyük risk, yapay zekanın fiş veya konuşmadan yanlış tutar çıkarmasıdır; bu risk her kaydın onaya sunulmasıyla yönetilir. Diğer riskler ve önlemler aşağıda listelenmiştir.

Risk	Etkisi	Çözüm yaklaşımı
Bulanık, buruşuk veya termal baskısı solmuş fiş	OCR metni eksik ya da hatalı olur	Onay kartında alanlar düzenlenebilir; okunamazsa kullanıcıdan yeniden çekim istenir
Modelin yanlış tutar veya kategori çıkarması	Hatalı kayıt bütçeyi bozar	Kaydetmeden önce zorunlu onay; model yanıtı JSON şemasıyla doğrulanır
"Yüz elli" ve "150" gibi farklı sayı ifadeleri	Sesli girişte tutar yanlış anlaşılır	Konuşma metni modele gönderilir; tutar onay kartında açıkça gösterilir
İnternet bağlantısının olmaması	Asistan yanıt veremez	Manuel işlem ekranına yönlendirme; diğer modüller çevrimdışı çalışır
API kotası veya ücretsiz kullanım sınırı	İstekler reddedilir	Hata mesajı ve manuel girişe geçiş; gereksiz model çağrılarından kaçınma
Kamera veya mikrofon izninin reddedilmesi	Fiş ve ses özelliği kullanılamaz	İznin neden gerektiği açıklanır; yazılı sohbet ve manuel giriş açık kalır

Test Yaklaşımı

Testler, Flutter'ın yerleşik test araçlarıyla üç seviyede ve gerçek fişlerle yapılan saha denemeleriyle yürütülür.

- **Birim testleri:** Bütçe hesaplama, kalan bütçe, limit aşımı kontrolü ve model yanıtından gelen JSON'un ayrıştırılması test edilir. Yapay zeka servisi sahte (mock) bir sınıfla değiştirilerek bağlantısız çalıştırılır.
- **Veritabanı testleri:** İşlem ekleme, güncelleme, silme ve tarih/kategori filtreli rapor sorguları bellek içi SQLite ile doğrulanır.
- **Widget testleri:** Onay kartının doğru alanları göstermesi, manuel formun zorunlu alan kontrolü ve Dashboard kartlarının doğru toplamları yansıtması sınanır.
- **Saha denemeleri:** Farklı marketlerden toplanan gerçek fişler ve örnek sesli komutlarla okunan tutarın doğruluğu elle kontrol edilir. Sonuçlar fiş türü ve hata nedeni ile birlikte kaydedilir.
- **Cihaz testleri:** Uygulama en az bir Android ve mümkünse bir iOS cihazda, izin akışları ve çevrimdışı senaryo dahil denenir.

Sonuç ve Beklenen Çıktılar

CepteCüzdan, harcama kaydını form doldurmaktan bir onay dokunuşuna indirerek bütçe takibini sürdürülebilir kılmayı hedefler. Proje, Mobil Programlama dersinde işlenen yerel veritabanı, durum yönetimi, cihaz donanımı erişimi ve harici servis entegrasyonu konularını tek bir uygulamada bir araya getirir.

Proje sonunda aşağıdaki çıktılar teslim edilecektir:

- Android (ve mümkünse iOS) üzerinde çalışan, dört modülü tamamlanmış Flutter uygulaması
- Fiş fotoğrafı, sesli komut ve yazılı sohbetle onaylı harcama ekleme özelliği
- Uygulamanın kaynak kodu ve SQLite veritabanı şeması

Kaynaklar

- google_mlkit_text_recognition – pub.dev
- speech_to_text – pub.dev
- firebase_ai (Firebase AI Logic) – pub.dev
- google_generative_ai (kullanımdan kaldırılmış) – pub.dev